

# Python Practice Handout

## *Dictionaries*

---

### 1. What is a dictionary

A dictionary is a cousin of the list. They are related, but not the same. The name comes from a real dictionary, where you look up a word and get its definition.

In Python the word you look up is called the key, and what you get back is called the value. Together they are a key and value pair.

### 2. Making a dictionary

Lists use square brackets. Dictionaries use curly brackets. Write the key, a colon, then the value, and separate each pair with a comma.

Here the cats' names are the keys and their ages are the values.

```
cats = {"Jane": 6, "Tom": 14, "Sara": 8}
```

This dictionary has three entries. The keys are strings and the values are ints.

### 3. Printing a dictionary

Printing the whole thing looks much like what you typed.

```
cats = {"Jane": 6, "Tom": 14, "Sara": 8}
print(cats)
```

Output:

```
{'Jane': 6, 'Tom': 14, 'Sara': 8}
```

### 4. Looking up a value

Put square brackets after the name, as you would with a list, but write the key instead of a number. You get back that key's value.

```
cats = {"Jane": 6, "Tom": 14, "Sara": 8}
print(cats["Tom"])
print(cats["Jane"])
```

Output:

```
14
6
```

### 5. The key must match exactly

Spelling and capital letters have to be right. A lowercase tom is not the same key as Tom, so Python cannot find it.

This does not work:

```
print(cats["tom"])
```

Error:

```
KeyError: 'tom'
```

## 6. Keys only, not values

This is a one way street. You can ask for a key and get its value, but you cannot ask for a value and get its key. Looking for the cat aged 8 gives an error.

This does not work:

```
print(cats[8])
```

Error:

```
KeyError: 8
```

## 7. Adding a new entry

Write the dictionary name with a new key in square brackets, then set it equal to a value. Here is a new cat named Wilson, aged one.

```
cats = {"Jane": 6, "Tom": 14, "Sara": 8}

cats["Wilson"] = 1

print(cats)
```

Output:

```
{'Jane': 6, 'Tom': 14, 'Sara': 8, 'Wilson': 1}
```

## 8. Removing an entry

Use del with the key, just as you did with lists. This removes both the key and its value.

```
cats = {"Jane": 6, "Tom": 14, "Sara": 8, "Wilson": 1}

del cats["Tom"]

print(cats)
```

Output:

```
{'Jane': 6, 'Sara': 8, 'Wilson': 1}
```

## 9. How many entries

len works on dictionaries too. It counts the pairs, not the keys and values separately.

```
cats = {"Jane": 6, "Tom": 14, "Sara": 8}

print(len(cats))
```

Output:

```
3
```

## 10. Spacing is up to you

In lists and dictionaries you can put a space after each comma or leave them out. Both run exactly the same way. A space after each comma is easier to read.

```
cats = {"Jane": 6, "Tom": 14}  
cats = {"Jane":6,"Tom":14}
```

## 11. Try it yourself

Make a dictionary that uses ints for the keys and Booleans for the values. Name it whatever you like. Try this on your own before looking below.

```
ints_and_bools = {5: True, 7: False, 144: False}  
  
print(ints_and_bools)  
print(ints_and_bools[5])
```

Output:

```
{5: True, 7: False, 144: False}  
True
```